

Arm 虚拟硬件实践专题二：Arm 虚拟硬件 FVP 模型入门指南

基础篇

Arm 虚拟硬件镜像（百度智能云版）在百度智能云上基于 Arm 架构的云服务器里提供了一个 Ubuntu Linux 镜像环境。镜像中包括用于物联网、机器学习和嵌入式应用程序的 Arm 开发工具，例如 Arm 编译器、FVP 模型和其他针对 Arm® Cortex® 系列处理器的开发工具。本文将重点向开发者展示如何在 Arm 虚拟硬件镜像环境中使用 FVP 模型进行软件算法功能的验证。

内容概要：

- 一. [Arm 虚拟硬件 FVP 模型简介](#)
- 二. [快速入门指南](#)

一. Arm 虚拟硬件 FVP 模型简介

Arm [固定虚拟平台](#) (Fixed Virtual Platform, FVP) 基于 Arm [快速模型](#) (Fast Model) 建模技术，是 Arm 系统的完整模拟，包含了相应的处理器、内存和外设。Arm 虚拟硬件 BCC 镜像中主要提供了面向 Arm Cortex-M 处理器的参考设计平台的 FVP 模型，是 Arm FVP 模型的子集，同时支持虚拟接口拓展的功能。关于如何使用 FVP 模型的虚拟接口，敬请期待下一篇指南《Arm 虚拟硬件实践专题三：虚拟接口应用指南》。

目前，Arm 虚拟硬件 BCC 镜像中提供了以下模型供用户试用（如何查看当前版本镜像中的 FVP 模型的方法详见上一篇指南《[Arm 虚拟硬件实践专题一：产品订阅指南（百度智能云版）](#)》的步骤三。

FVP 仿真模型	处理器 IP	技术帮助文档
VHT_MPS2_Cortex-M0	Cortex-M0	ARM Cortex-M0 SMM on V2M-MPS2 (AppNote AN382)
VHT_MPS2_Cortex-M0plus	Cortex-M0+	ARM Cortex-M0+ SMM on V2M-MPS2 (AppNote AN383)
VHT_MPS2_Cortex-M3	Cortex-M3	ARM Cortex-M3 SMM on V2M-MPS2 (AppNote AN385)
VHT_MPS2_Cortex-M4	Cortex-M4	ARM Cortex-M4 SMM on V2M-MPS2 (AppNote AN386)
VHT_MPS2_Cortex-M7	Cortex-M7	ARM Cortex-M7 SMM on V2M-MPS2 (AppNote AN399)
VHT_MPS2_Cortex-M23	Cortex-M23	ARM Cortex-M23 IoT Subsystem for V2M-MPS2+ (AppNote AN519)
VHT_MPS2_Cortex-M33	Cortex-M33	ARM Cortex-M33 IoT Subsystem for V2M-MPS2+ (AppNote AN505)
VHT_MPS3_Corstone_SSE-300	Cortex-M55	Corstone-300 FVP Technical Overview (PDF), Memory map overview
eVHT_Corstone_SSE-300_Ethos-U55	Cortex-M55, Ethos-U55	Corstone-300 FVP Technical Overview (PDF), Memory map overview

FVP 仿真模型	处理器 IP	技术帮助文档
VHT_Corstone_SSE-300_Ethos-U65	Cortex-M55, Ethos-U65	Corstone-300 FVP Technical Overview (PDF) , Memory map overview
VHT_Corstone_SSE-310	Cortex-M85, Ethos-U55	Corstone-310 FVP Technical Overview (PDF) , Memory map overview
VHT_Corstone_SSE-310_Ethos-U65	Cortex-M85, Ethos-U65	Corstone-310 FVP Technical Overview (PDF) , Memory map overview

表：FVP 模型清单及帮助文档

二. 快速入门指南

1. 运行用户应用程序

Arm 虚拟硬件 BCC 镜像中(基础操作系统为 Linux) 可以通过命令行的形式来调用目标平台的 FVP 模型验证构建完成的应用程序。Arm 虚拟硬件 BCC 镜像中集成了 [CMSIS-Toolbox](#) 等软件工具能够帮助开发者轻松地构建和管理工程（工程符合 [Open-CMSIS-Pack](#) 标准可以轻松实现不同目标平台的移植）。用户可根据自身开发习惯选择通过本地集成开发环境（例如: Keil MDK）或云服务器环境（Ubuntu Linux）进行应用程序的构建。

一个简单的运行示例如下：

```
VHT_Corstone_SSE-300_Ethos-U55 -a microspeech.Helium.axf -V "/VSI/audio/python" -f fvp_config.txt --stat --simlimit 24
```

其中，

- VHT_Corstone_SSE-300_Ethos-U55 是目标平台 FVP 模型的名称。
- -a (或--application) 参数指定在目标平台上运行的用户应用的二进制镜像文件 (此示例中为 "microspeech.Helium.axf")。
- -V (或--v_path) 参数指定虚拟接口定义的 Python 脚本的路径此示例中为 ("/VSI/audio/python")。
- -f (或--config-file) 参数指定目标平台的具体配置 (此示例中为 "fvp_config.txt")，详细介绍请参考下一小节《模型配置》。
- --stat 参数设定在仿真结束后需要打印相关仿真运行参数信息。
- --simlimit 参数指定仿真运行的最长时间(秒数)。

若想查看所有支持的命令行参数，可以使用 --help 参数 (VHT_Corstone_SSE-300_Ethos-U55 --help) 获取提示。更详细完整的参数说明请参考《[FVP 命令行帮助文档](#)》。

2. 模型配置

Arm 虚拟硬件 BCC 镜像中的 FVP 模型是通过预先构建好的可执行文件提供给开发者进行试用。这些 FVP 模型的系统组成是固定的，但可以通过配置模型参数来具体控制它们的行为。

调用目标平台的 FVP 模型时，可通过以下命令行参数配置 FVP 模型的行为：

- 使用 `-C` (或 `--parameter`) 选项配置单个参数。例如：`VHT_Corstone_SSE-300_Ethos-U55 -C core_clk.mul=100000000`
- 使用 `-f` (或 `--config-file`) 选项指定含有配置参数的文本文件的路径。例如：`VHT_Corstone_SSE-300_Ethos-U55 -f fvp_config.txt`

但是，请注意具体的配置参数是特定于模型的（不同的 FVP 模型相应的参数有所差异），并且按照 `instance.parameter=<value>` 的格式指定。其中，`instance` 参数指定了仿真的实例（组件）目标，例如：CPU、接口、总线、存储器、外设等，同时也支持分层定义（使用 `"` 进行分层）。查看目标平台的 FVP 模型全部参数的默认值及其详细说明，可以使用 `-l` 或者 `--list-params` 参数获取。因参数总量较多，建议用户可以通过以下命令将所有参数的信息保存至文本文件，方便查阅和修改定义。

```
VHT_Corstone_SSE-300_Ethos-U55 --list-params > config.txt
```

一个简单的配置文件示例如下：

```
# Parameters:
# instance.parameter=value
#(type, mode) default = 'def value': description : [min..max]
#-----
core_clk.mul=100000000
# (int, init-time) default = '0x17d7840': Clock Rate Multiplier. This parameter is not exposed via CADI and can
only be set in LISA
mps3_board.telnetterminal0.start_telnet=0
# (bool, init-time) default = '1': Start telnet if nothing connected
mps3_board.uart0.out_file=-
# (string, init-time) default = '': Output file to hold data written by the UART (use '-' to send all output to stdout)
mps3_board.visualisation.disable-visualisation=1
# (bool, init-time) default = '0': Enable/disable visualization
cpu0.semihosting-enable=1
# (bool, init-time) default = '0': Enable semihosting SVC traps. Applications that do not use semihosting must
set this parameter to false.
#-----
```

其中，

- `core_clk.mul=100000000` 指定了虚拟时间，此处是 100MHz CPU 时钟。
- `mps3_board.telnetterminal0.start_telnet=0` 禁用了 Telnet 连接。
- `mps3_board.uart0.out_file=-` 采用标准 UART 输出。
- `mps3_board.visualisation.disable-visualisation=1` 禁用了 FVP 的图形化用户接口。
- `cpu0.semihosting-enable=1` 启动了 FVP 模型的半主机 (Semihosting) 功能。

此外，通过命令行参数 `--list-instance`（例如：`VHT_Corstone_SSE-300_Ethos-U55 --list-instance`）可以获取目标平台 FVP 模型中实现模拟的实例（组件）目标。该选项将返回实例（组件）目标的名称和其相应的快速模型（Fast Model）组件类型、版本信息和简要描述。更完善的快速模型组件介绍请参考《[快速建模组件帮助文档](#)》。

本节仅介绍了部分 FVP 模型的参数配置方法，更全面的配置选项和参数说明请参考《[FVP 模型快速入门指南](#)》的[“模型配置”](#)章节。

3. 说明提示

本节将提供一些使用 FVP 模型的小贴士，用户可根据开发需求选择性阅读和使用。

3.1. 结束仿真

可以通过模型配置参数来控制仿真程序的停止。FVP 模型具有 `shutdown_on_eot` 等参数来控制退出程序执行的功能。例如，对于 VHT_Corstone_SSE-300 模型，可配置：

```
mps3_board.uart0.shutdown_on_eot=1
#(bool,init-time) default = '0': Shutdown simulation when a EOT (ASCII 4) char is transmitted
(useful for regression tests when semihosting is not available)
```

或配置：

```
mps3_board.uart0.shutdown_tag="EXITTHESIM"
#(string,run-time) default = "": Shutdown simulation when a string is transmitted
```

对于情况一，即配置 `mps3_board.uart0.shutdown_on_eot=1` 时，当串口 UART0 接收到 EOT (ASCII 4) 字符时，将会退出仿真模拟。因此，用户只需要在主程序源码中包含输出 EOT 字符至串口 UART0 的代码即可轻松地停止程序的运行。例如，以下示例中通过打印输出 0x04 (EOT) 至串口 UART0 即可停止模拟仿真的运行（该示例代码的其余部分中实现了由标准输出到串口 UART0 的重定向）。

```
while (1) {
    printf ("Hello World %d\r\n", count);
    if (count > 100) printf ("\x04"); // EOT (0x04) stops simulation.
    count++;
    osDelay (1000);
}
```

类似地，对于情况二，即配置 `mps3_board.uart0.shutdown_tag="EXITTHESIM"` 时，当串口 UART0 接收到指定的字符串 EXITTHESIM 即可停止仿真运行。

3.2. 获取时序参数

需注意，FVP 仿真模型主要面向软件开发和功能测试，并不适用于 CPU 层面的准确的性能比较。但是，它们可以很好地用于分析程序级别的时序，例如 RTOS 的任务调度、死锁检测，以及软件总体性能趋势的判断。下面的原理和配置可用于时序的控制和测量：

- a. `--stat` 参数：FVP 模型可以使用命令行参数 `--stat` 来显示仿真退出时运行时间统计信息。一个显示示例如下：

```
----- cpu_core statistics: -----
Simulated time                               :0.651964s
```

```
User time           : 0.843750s
System time        : 0.109375s
Wall time          : 1.503559s
Performance index   : 0.43
cpu_core.cpu0      : 68.40 MIPS ( 65196784 Inst)
```

统计信息参数的详细说明请参考《[FVP 命令行选项](#)》以及《[快速模型用户指南](#)》的相关章节“[显示模拟的总执行时间](#)”。

- b. [CMSIS-View](#) 工具包可以用于测量和分析程序中事件之间的时序，包括执行时间等统计数据。请注意，如需在 FVP 上存储日志信息，半主机（Semihosting）功能需要被启用。CMSIS-View 的注释也可复用于真实物理开发板上的事件分析和测量。
- c. 模型配置参数也可用于控制和影响程序的执行时间：
 - i. 设置指令平均周期数（Cycle Per Instruction, CPI）`cpi_div` 和 `cpi_mul`。
 - ii. 设置时钟相关参数。例如：时钟频率 Clock Rate Multiplier 可通过配置 CS300 FVP 模型的 `core_clk.mul` 参数。
 - iii. 设置影响流水线的相关参数。例如：内存缓存（memory cache），浮点单元（FPU）或者矢量扩展技术（M-Profile Vector Extension, MVE）等。

《[快速模型用户指南](#)》的“[时序注释](#)”章节详细解释说明了在底层 Fast Model 技术中实现性能估计的方法。

3.3. 半主机功能（Semihosting）

半主机功能是一种机制，它使得在 FVP 模型上运行的代码能够直接访问主机上的输入/输出设备，例如控制台 I/O 和文件 I/O。这通常对于测试和调试软件很有用，特别是当模型中并非严格需要使用硬件的输入/输出接口，或者这样做会使程序变得不必要复杂时。

默认情况下，半主机功能在 FVP 中是处于禁用状态的。可以使用 CPU 的 `semihosting-enable` 参数来启用。例如，对于 VHT_Corstone_SSE-300/310 平台，可以配置：

```
cpu0.semihosting-enable=1
```

或者对于 VHT_MPS2_Cortex-M4 平台，配置：

```
armcortexm4ct.semihosting-enable=1
```

半主机功能对于以下场景非常有用：

- a. 半主机模式下的 FVP 模型可以与 CMSIS-View 工具包搭配使用，将事件日志存储到主机上的文件中。详细方法请参考《[事件记录器 Event Recorder](#)》指南中的“[半主机](#)”章节。
- b. 调用 `stdio.h` 函数时直接使用主机平台。例如，这可以绕过通过 UART 进行消息重定向。